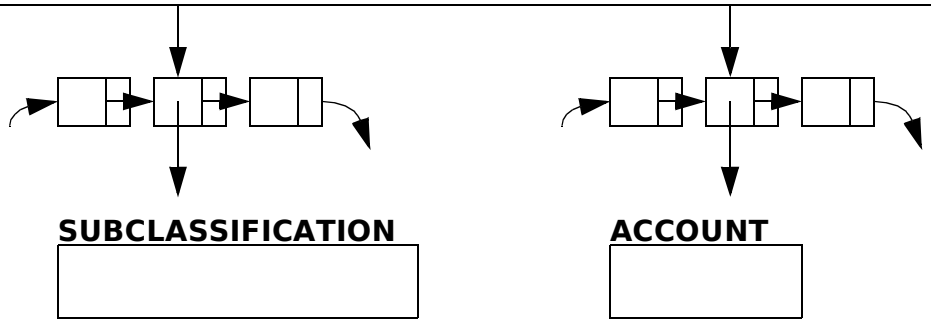
# PredictiveBooks Data Structure

## ELEMENT

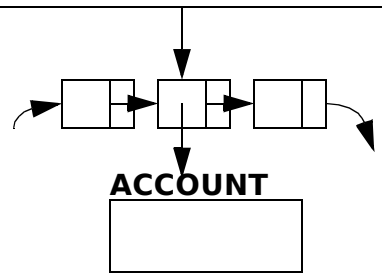
```
/* Usage */
LIST *element_list(
        LIST *filter_element_name_list,
        char *fund_name,
        char *transaction_date_time_closing(),
        boolean fetch_subclassification_list,
        boolean fetch_account_list );

/* Attributes */
char *element_name;
boolean accumulate_debit;
LIST *subclassification_list;
LIST *account_list;

/* Process */
double element_total(
        subclassification_list,
        account_list );

int percent_of_assets;
int percent_of_revenues;

void element_list_set_account_action_string(
        LIST *element_list,
        char *application_name,
        char *session,
        char *login_name,
        char *role_name,
        char *begin_date_string,
        char *as_of_date );

void element_list_set_total(
        LIST *element_list );

void element_list_set_percent_of_assets(
        LIST *element_list );

void element_list_set_percent_of_revenues(
        LIST *element_list );

void element_denominator_set_percent_of_assets(
        LIST *element_list,
        double assets_total );

void element_denominator_set_percent_of_revenues(
        LIST *element_list,
        double revenues_total );

void element_list_set_delta_prior(
        LIST *prior_year_element_list,
        LIST *preclose_element_list );

void element_prior_year_element_list_set_delta_prior(
        LIST *prior_year_element_list,
        ELEMENT *preclose_element );

int element_delta_prior(
        double prior_total,
        double total );

double element_list_debit_total(
        LIST *element_list );

double element_list_credit_total(
        LIST *element_list );

ACCOUNT *element_list_account_seek(
        char *account_name,
        LIST *element_list );
```




Tim Riley © 2012-2021

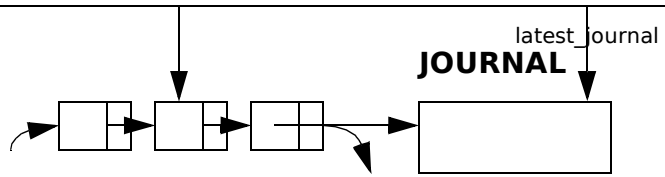
## SUBCLASSIFICATION

```
/* Usage */
SUBCLASSIFICATION *subclassification_total_fetch(
        double *subclassification_total,
        char *subclassification_name,
        char *fund_name,
        char *transaction_date_time_closing() );

/* Attributes */
char *subclassification_name;

/* Process */
LIST *account_list;
int percent_of_total;

LIST *account_list =
        subclassification_total_account_list(
            &subclassification_total,
            subclassification_name,
            fund_name,
            transaction_date_time_closing() );

double subclassification_list_total(
        subclassification_list );

double subclassification_total =
        account_list_total(
            account_list );

void subclassification_list_set_percent_of_assets(
        LIST *subclassification_list,
        double assets_total );

void subclassification_list_set_percent_of_revenues(
        LIST *subclassification_list,
        double revenues_total );

void subclassification_list_set_delta_prior(
        LIST *prior_subclassification_list,
        LIST *subclassification_list );

void subclassification_set_delta_prior(
        LIST *prior_subclassification_list,
        SUBCLASSIFICATION *subclassification );

int delta_prior =
        element_delta_prior(
            prior_subclassification->subclassification_total,
            subclassification->subclassification_total );

double subclassification_list_debit_total(
        LIST *subclassification_list );

double subclassification_list_credit_total(
        LIST *subclassification_list );
```



## ACCOUNT

```
/* Attributes */
char *account_name;
char *fund_name;
char *subclassification_name;
char *hard_coded_account_key;
boolean accumulate_debit;
double balance;
double payment_amount;

/* Process */
LIST *journal_list;
JOURNAL *latest_journal;
double account_total;
int percent_of_assets;
int percent_of_revenues;

char *account_action_string(
        char *application_name,
        char *session,
        char *login_name,
        char *role_name,
        char *begin_date_string,
        char *as_of_date,
        char *account_name );

double account_list_total(
        LIST *account_list );

void account_list_set_percent_of_assets(
        LIST *account_list,
        double assets_total );

void account_list_set_percent_of_revenues(
        LIST *account_list,
        double revenues_total );

void account_list_set_delta_prior(
        LIST *prior_account_list,
        LIST *account_list );

void account_set_delta_prior(
        LIST *prior_account_list,
        ACCOUNT *account );

int delta_prior =
        element_delta_prior(
            prior_account->
                latest_journal->
                balance,
            account->
                latest_journal->
                balance );

double account_list_debit_total(
        LIST *account_list );

double account_list_credit_total(
        LIST *account_list );
```

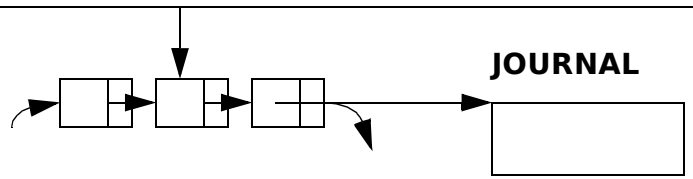


## JOURNAL

```
char *full_name;
char *street_address;
char *transaction_date_time;
char *account_name;
double previous_balance;
double previous_balance_database;
double debit_amount;
double credit_amount;
double balance;
int transaction_count;
boolean match_sum_taken;
char *transaction_memo;

JOURNAL *journal_new(
        full_name,
        street_address,
        transaction_date_time,
        account_name );

char *journal_select(
        void );

JOURNAL *journal_parse(
        input );

LIST *journal_list(
        full_name,
        street_address,
        transaction_date_time );

LIST *journal_list_fetch(
        where_clause );

JOURNAL *journal_prior(
        transaction_date_time,
        account_name );
FILE *journal_insert_open(
        void );

void journal_insert(
        insert_pipe,
        full_name,
        street_address,
        transaction_date_time,
        account_name,
        amount,
        is_debit );

/* Also does journal_propagate() for each account */
void journal_delete(
        full_name,
        street_address,
        transaction_date_time );

void journal_propagate(
        transaction_date_time,
        account_name );

LIST *journal_list_minimum(
        minimum_transaction_date_time,
        account_name );

LIST *journal_list_account(
        account_name );

void journal_list_set_balances(
        journal_list_prior(),
        account_accumulate_debit(
            account_name ) );

void journal_list_propagate_update(
        journal_list_prior() );

JOURNAL *journal_account_fetch(
        account_name,
        transaction_date_time );

LIST *journal_account_name_list(
        full_name,
        street_address,
        transaction_date_time );

LIST *journal_binary_list(
        full_name,
        street_address,
        transaction_date_time,
        transaction_amount,
        debit_account,
        credit_account );

LIST *journal_year_list(
        int year,
        char *account_name,
        boolean fetch_memo );
```

## TRANSACTION 1

```
char *full_name;
char *street_address;
char *transaction_date_time;
double transaction_amount;
char *memo;
int check_number;
boolean lock_transaction;
char *property_street_address;
char *program_name;
LIST *journal_list;

TRANSACTION *transaction_calloc(
        void );

TRANSACTION *transaction_new(
        full_name,
        street_address,
        transaction_date_time );

TRANSACTION *transaction_fetch(
        full_name,
        street_address,
        transaction_date_time );

char *transaction_primary_where(
        full_name,
        street_address,
        transaction_date_time );

void transaction_refresh(
        full_name,
        street_address,
        transaction_date_time,
        transaction_amount,
        memo,
        check_number,
        lock_transaction,
        journal_list );

/* Returns inserted transaction_date_time */
char *transaction_insert(
        full_name,
        street_address,
        transaction_date_time,
        transaction_amount,
        memo,
        check_number,
        lock_transaction );

FILE *transaction_insert_pipe_open(
        void );

/* Returns inserted transaction_date_time */
char *transaction_insert_pipe(
        insert_pipe,
        full_name,
        street_address,
        transaction_date_time,
        transaction_amount,
        memo,
        check_number,
        lock_transaction,
        journal_list );

/* Returns transaction_list */
LIST *transaction_list_insert(
        transaction_list );

void transaction_list_journal_insert(
        transaction_list );

void transaction_update(
        transaction_amount,
        full_name,
        street_address,
        transaction_date_time );

char *transaction_select(
        void );

char *transaction_primary_where(
        full_name,
        street_address,
        transaction_date_time );

TRANSACTION *transaction_parse(
        input_buffer );

/* Performs journal_propagate() */
void transaction_delete(
        full_name,
        street_address,
        transaction_date_time );

TRANSACTION *transaction_seek(
        transaction_list,
        full_name,
        street_address,
        transaction_date_time );

/* Returns race-free transaction_date_time */
char *transaction_race_free(
        transaction_date_time );
```



## TRANSACTION 2

```
TRANSACTION *transaction_binary(
        full_name,
        street_address,
        transaction_date_time,
        double transaction_amount,
        char *memo,
        char *debit_account,
        char *credit_account );

char *transaction_memo(
        memo );

void transaction_check_insert(
        insert_pipe,
        check_number );

void transation_lock_insert(
        insert_pipe,
        lock_transaction );

boolean transaction_exists(
        transaction_date_time );

char *transaction_display(
        transaction );

char *transaction_escape_full_name(
        full_name );

TRANSACTION *transaction_check_seek(
        transaction_list,
        check_number );

boolean transaction_closing_entry_exists(
        char *transaction_date );

char *transaction_date_time_closing(
        char *transaction_date,
        boolean preclose_time,
        boolean transaction_closing_entry_exists() );
```

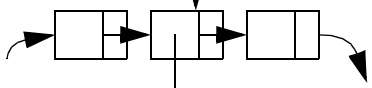
## STATEMENT

```
/* Usage */
STATEMENT *statement_new(
        char *application_name,
        char *session,
        char *login_name,
        char *role_name,
        char *process_name,
        boolean exists_logo_filename,
        LIST *filter_element_name_list,
        char *as_of_date,
        int prior_year_count,
        LIST *fund_name_list,
        char *subclassification_option_string,
        char *fund_aggregation_string,
        char *output_medium_string );

STATEMENT *statement_steady_state(
        char *application_name,
        char *session,
        char *login_name,
        char *role_name,
        char *process_name,
        boolean exists_logo_filename,
        LIST *filter_element_name_list,
        char *as_of_date,
        int prior_year_count,
        LIST *fund_name_list,
        char *subclassification_option_string,
        char *fund_aggregation_string,
        char *output_medium_string,
        STATEMENT *statement );

enum subclassification_option
        statement_subclassification_option(
            subclassification_option_string );

enum fund_aggregation
        statement_fund_aggregation(
            fund_name_list,
            fund_aggregation_string );

enum output_medium
        statement_output_medium(
            output_medium_string );

LIST *statement_fund_name_list( void );

char *statement_begin_date_string(
        char *as_of_date );

char *statement_title(
        char *application_name,
        boolean exists_logo_filename,
        char *process_name );

char *statement_subtitle(
        statement_begin_date_string(),
        char *as_of_date,
        enum fund_aggregation );

LIST *statement_fund_list(
        application_name,
        session,
        login_name,
        role_name,
        filter_element_name_list,
        statement_begin_date_string(),
        as_of_date,
        prior_year_count,
        fund_name_list,
        statement_subclassification_option(),
        statement_fund_aggregation() );

boolean statement_exists_closing_entry(
        char *as_of_date );
```

subclassification_aggregate,
subclassification_display,
subclassification_omit

single_fund,
sequential,
consolidated,
no_fund

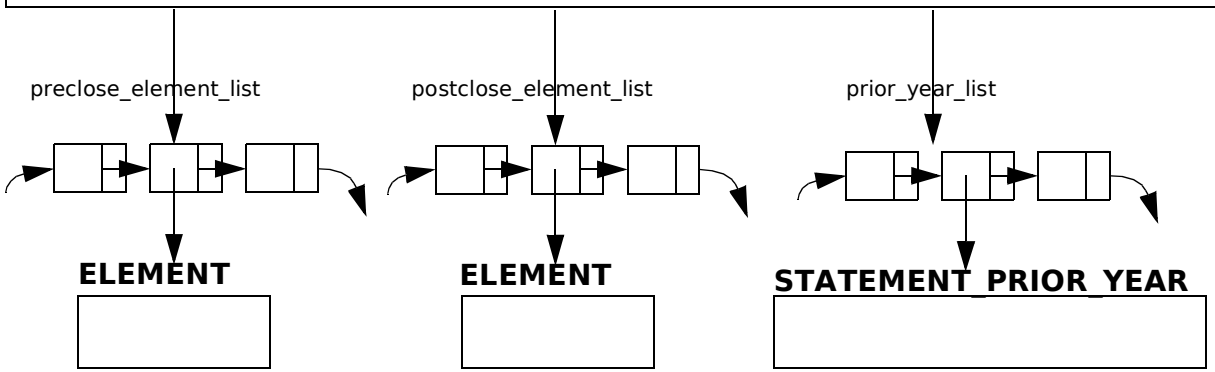
output_table,
output_PDF
output_stdout



**STATEMENT_FUND**

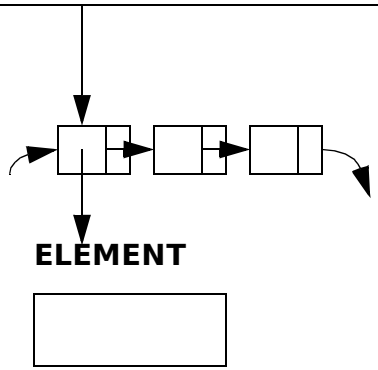
## STATEMENT_FUND

```
/* Usage */
STATEMENT_FUND *statement_fund_fetch(
    char *application_name,
    char *session,
    char *login_name,
    char *role_name,
    LIST *filter_element_name_list,
    char *begin_date_string,
    char *as_of_date,
    int prior_year_count,
    char *fund_name,
    enum subclassification_option );

LIST *statement_fund_element_list(
        char *application_name,
        char *session,
        char *login_name,
        char *role_name,
        LIST *filter_element_name_list,
        char *begin_date_string,
        char *transaction_date_time,
        char *fund_name,
        enum subclassification_option );

LIST *preclose_element_list =
        statement_fund_element_list(
            application_name,
            session,
            login_name,
            role_name,
            filter_element_name_list,
            begin_date_string,
            transaction_date_time_closing(
                as_of_date,
                1 /* preclose_time_boolean */ ),
            fund_name,
            subclassification_option );

LIST *postclose_element_list =
        statement_fund_element_list(
            application_name,
            session,
            login_name,
            role_name,
            filter_element_name_list,
            begin_date_string,
            transaction_date_time_closing(
                as_of_date,
                0 /* not preclose_time_boolean */ ),
            fund_name,
            subclassification_option );

LIST *prior_year_list;

for years_ago = 1 to prior_year_count
{
        list_set(
            prior_year_list,
            statement_prior_year_fetch(
                application_name,
                session,
                login_name,
                role_name,
                filter_element_name_list,
                as_of_date,
                years_ago,
                fund_name,
                subclassification_option ) );
}

void statement_fund_list_steady_state(
        LIST *statement_fund_list );

STATEMENT_FUND *statement_fund_steady_state(
        LIST *preclose_element_list,
        LIST *current_element_list,
        int prior_year_count,
        STATEMENT_FUND *statement_fund );

double preclose_debit_total =
        element_list_debit_total(
            preclose_element_list );

double preclose_credit_total =
        element_list_credit_total(
            preclose_element_list );

double postclose_debit_total =
        element_list_debit_total(
            postclose_element_list );

double postclose_credit_total =
        element_list_credit_total(
            postclose_element_list );

boolean statement_fund_exists_postclose(
        LIST *statement_fund_list );
```

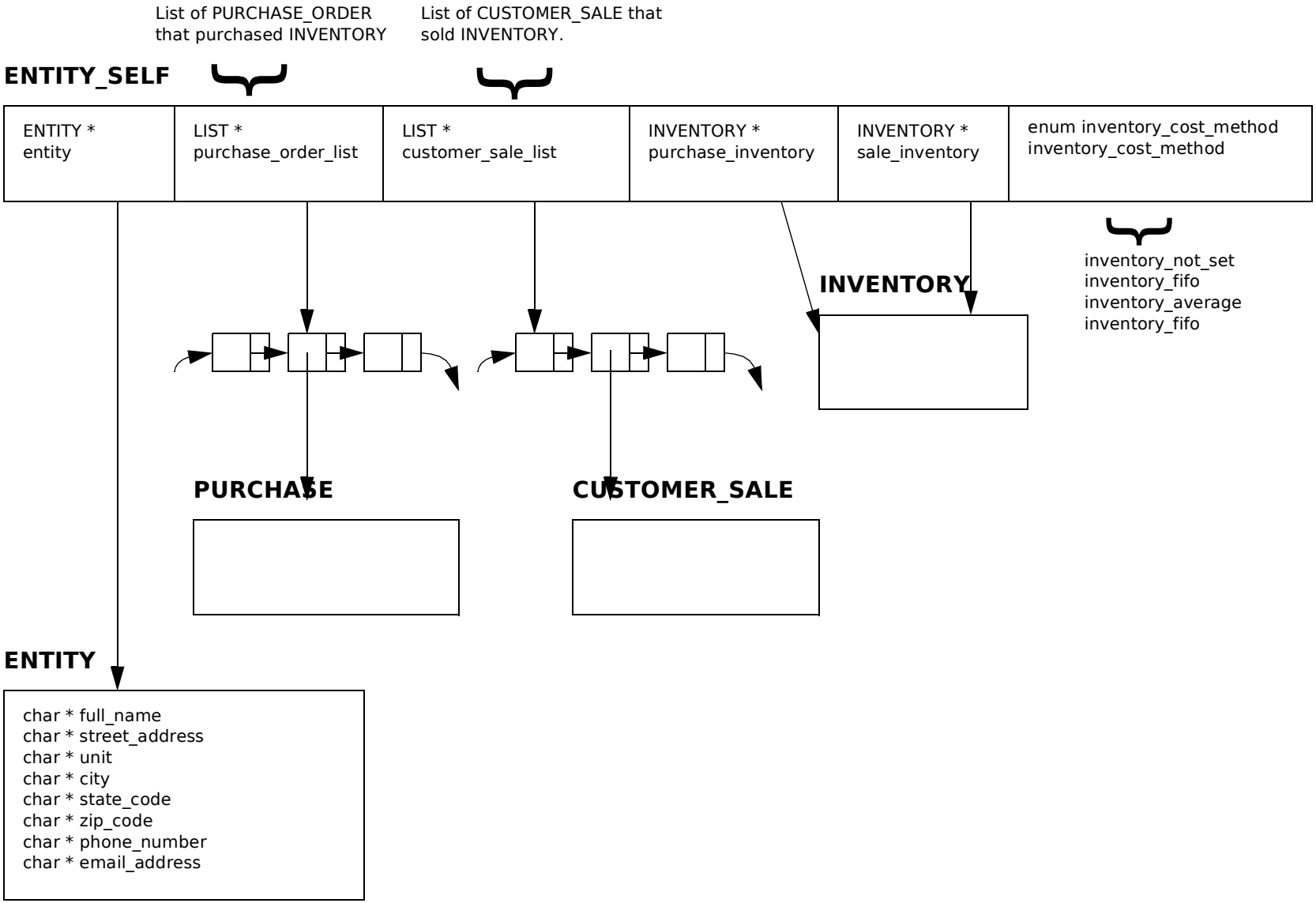


## STATEMENT_PRIOR_YEAR

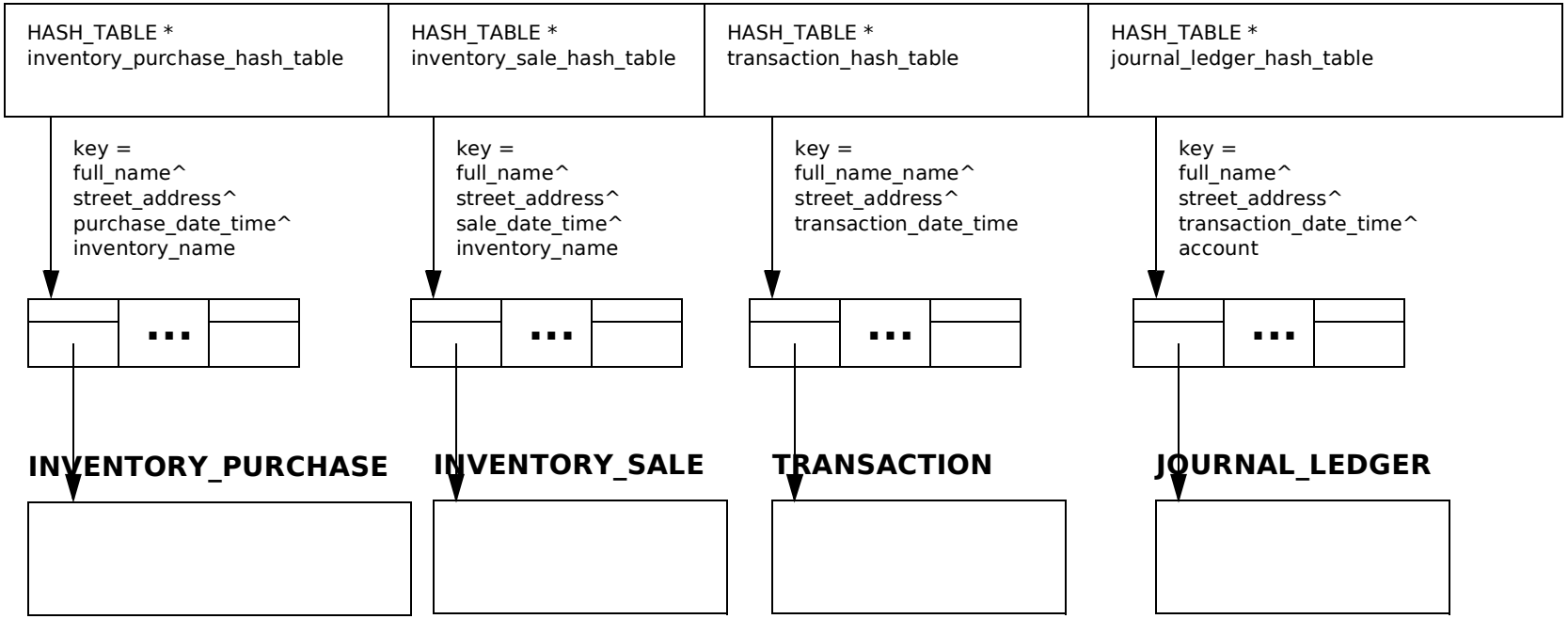
```
/* Usage */
STATEMENT_PRIOR_YEAR *
    statement_prior_year_fetch(
        char *application_name,
        char *session,
        char *login_name,
        char *role_name,
        LIST *filter_element_name_list,
        char *as_of_date,
        int years_ago,
        char *fund_name,
        enum subclassification_option );

/* Attributes */
char *application_name;
char *session;
char *login_name;
char *role_name;
LIST *filter_element_name_list;
char *fund_name;
enum subclassification_option subclassification_option;

/* Process */
char *prior_date_string;
LIST *prior_year_element_list =
    statement_element_list(
        application_name,
        session,
        login_name,
        role_name,
        filter_element_name_list,
        begin_date_string,
        strdup(
            transaction_date_time_closing(
                date_yyyy_mm_dd_display(
                    date_subtract_year(
                        date_yyyy_mm_dd_new(
                            as_of_date ),
                        years_ago ) ),
                0 /* not preclose_time_boolean */ ) ),
        fund_name,
        subclassification_option );

LIST *statement_prior_year_heading_list(
    LIST *prior_year_list );

LIST *statement_prior_year_delta_list(
    char *account_name,
    LIST *prior_year_list );
```



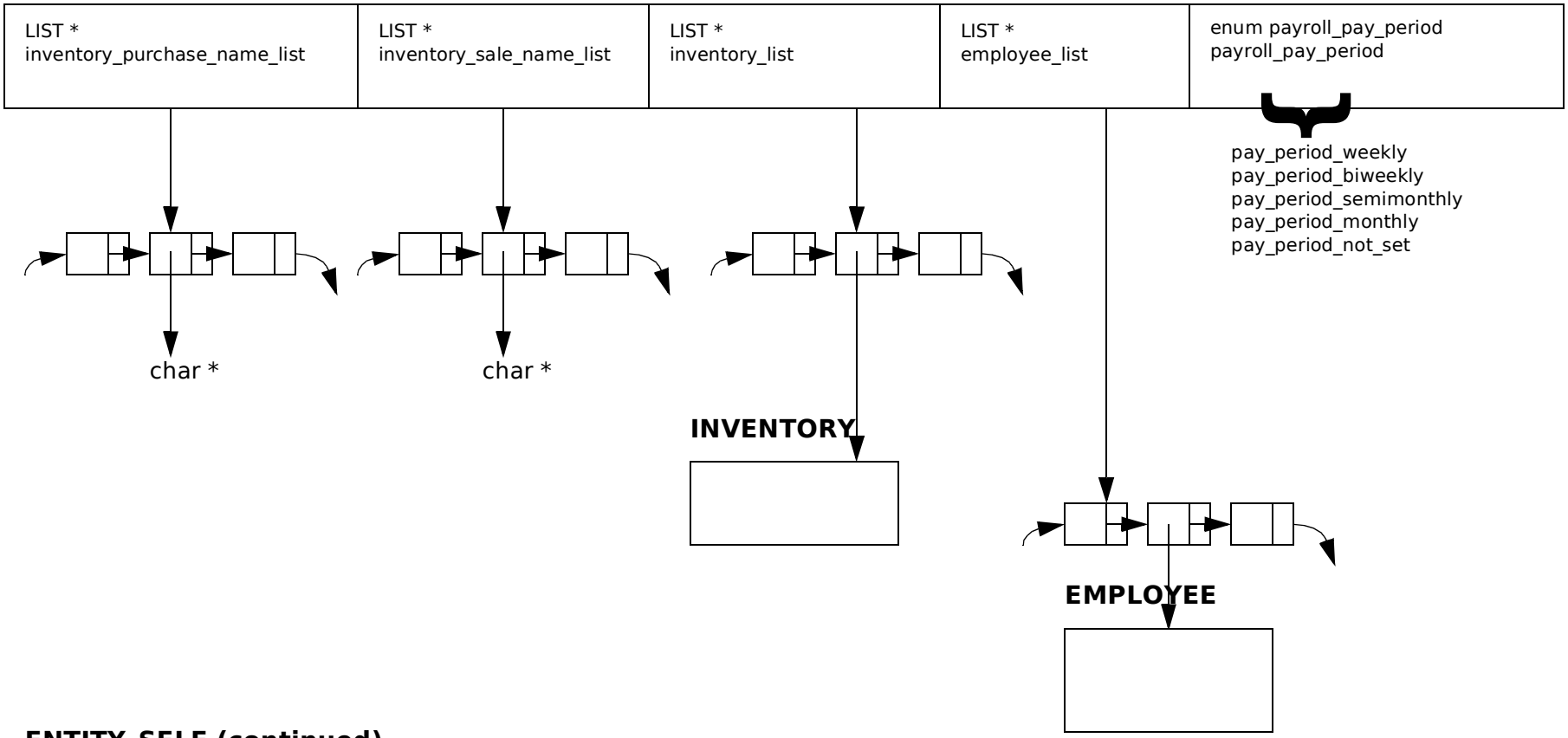
# PredictiveBooks Data Structure



### ENTITY_SELF (continued)



## ENTITY_SELF (continued)




## ENTITY_SELF (continued)

```
char *payroll_beginning_day
double social_security_combined_tax_rate
int social_security_payroll_ceiling
double medicare_combined_tax_rate
double medicare_additional_withholding_rate
int medicare_additional_gross_pay_floor
double federal_withholding_allowance_period_value
double federal_nonresident_withholding_income_premium
double state_withholding_allowance_period_value
double state_itemized_allowance_period_value
int federal_unemployment_wage_base
double federal_unemployment_tax_standard_rate
double federal_unemployment_threshold_rate
double federal_unemployment_tax_minimum_rate
int state_unemployment_wage_base
double state_unemployment_tax_rate
double state_sales_tax_rate;

char *entity_self_paypal_cash_account_name();
```

## SALE (1)

```
ENTITY *customer_entity =
        entity_fetch(
            full_name,
            street_address );

char *sale_date_time;
char *completed_date_time;
double shipping_revenue;
enum title_passage_rule title_passage_rule;
double sales_tax_database;
char *transaction_date_time_database;

/* Returns sale_steady_state() */
SALE *sale_fetch(
        full_name,
        street_address,
        sale_date_time );

char *sale_primary_where(
        full_name,
        street_address,
        sale_date_time );

double sale_gross_revenue(
        inventory_sale_total(),
        fixed_service_sale_total(),
        hourly_service_sale_total() );

double sale_sales_tax(
        inventory_sale_list(),
        entity_self_sales_tax_rate() );

double sale_invoice_amount(
        sale_gross_revenue(),
        sale_sales_tax(),
        shipping_charge );

double Sale_amount_due(
        sale_invoice_amount(),
        customer_payment_total() )
{
        return A - B;
}

TRANSACTION *sale_transaction(
        full_name,
        street_address,
        completed_date_time,
        invoice_amount,
        gross_revenue,
        sale_tax,
        shipping_charge,
        account_receivable(),
        account_revenue(),
        account_shipping_revenue(),
        account_sales_tax_payable() );

void sale_update(
        inventory_sale_total(),
        fixed_service_sale_total(),
        hourly_service_sale_total(),
        sale_gross_revenue()
        sale_sales_tax(),
        sale_invoice_amount(),
        payment_total(),
        sale_amount_due(),
        sale_transaction()->
            transaction_date_time,
        full_name,
        street_address,
        sale_date_time );

SALE *sale_steady_state(
        full_name,
        street_address,
        sale_date_time,
        inventory_sale_list(),
        fixed_service_sale_list(),
        hourly_service_sale_list(),
        customer_payment_list(),
        shipping_charge,
        entity_self_sales_tax_rate(),
        sales_tax_database,
        completed_date_time,
        transaction_date_time_database.
        account_receivable(),
        account_revenue(),
        account_shipping_revenue(),
        account_sales_tax_payable() );
```

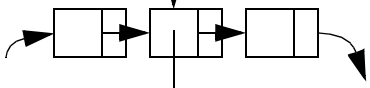
**TRANSACTION**

**ENTITY**

FOB_shipping
FOB_destination
title_passage_rule_null

## SALE (2)

```
LIST *customer_payment_list(
        full_name,
        street_address,
        sale_date_time );
```



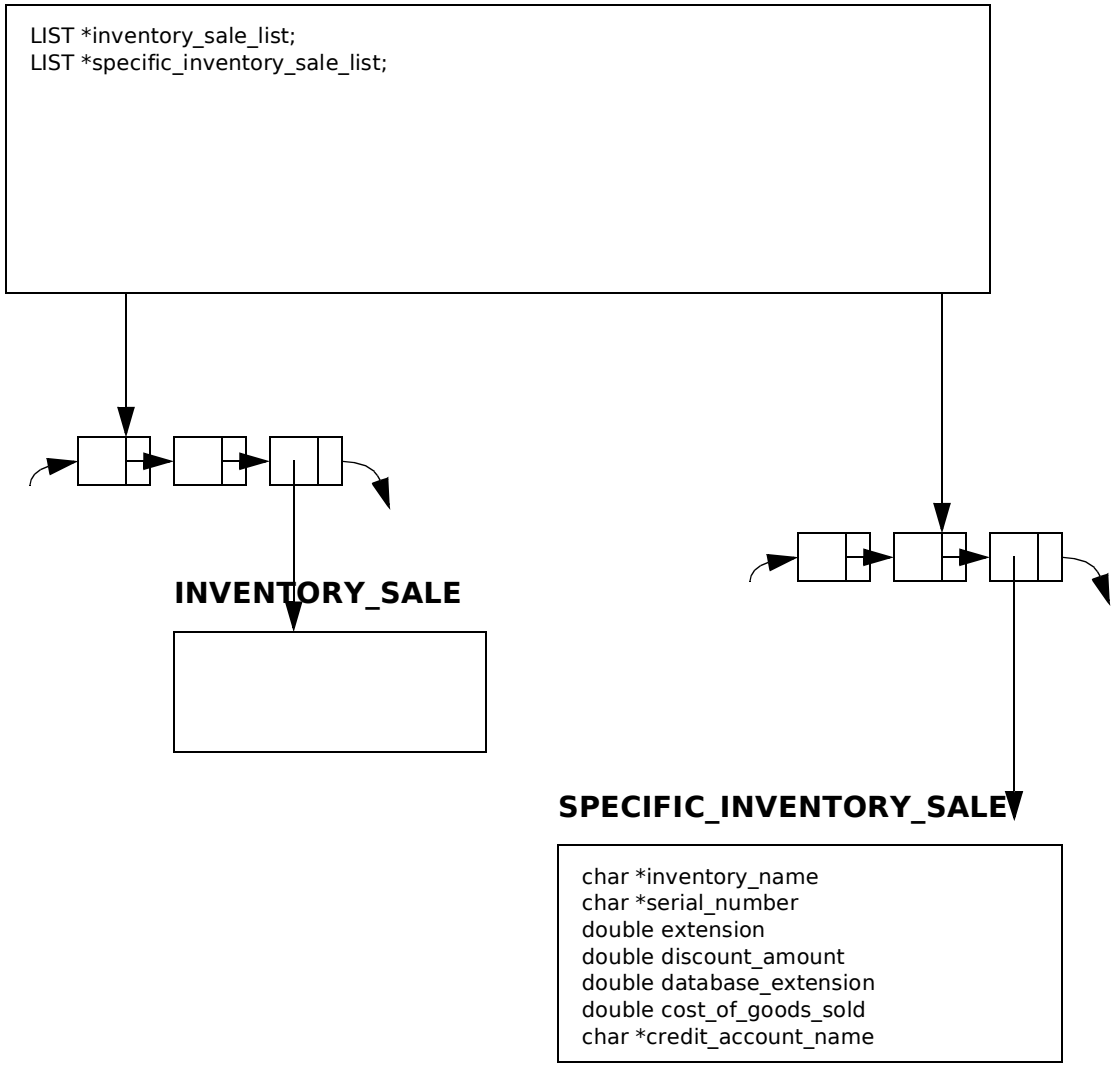
## CUSTOMER_PAYMENT

```
ENTITY *customer_entity =
        entity_fetch(
            full_name,
            street_address );

char *sale_date_time;
char *payment_date_time;
double payment_amount;
int check_number;

double customer_payment_total(
        customer_payment_list() )
{
        return sum( A->payment_amount );
}

CUSTOMER_PAYMENT *customer_payment_new(
        full_name,
        street_address,
        sale_date_time,
        payment_date_time );

char *customer_payment_select(
        void );

char *customer_payment_sys_string(
        where );

LIST *customer_payment_system_list(
        sys_string );

CUSTOMER_PAYMENT *customer_payment_parse(
        input );

TRANSACTION *customer_payment_transaction(
        full_name,
        street_address,
        payment_date_time,
        payment_amount,
        account_cash(),
        account_receivable() );
```

## ENTITY

## TRANSACTION

**SALE (3)**


LIST *inventory_sale_list;
LIST *specific_inventory_sale_list;
INVENTORY_SALE
SPECIFIC_INVENTORY_SALE
char *inventory_name
char *serial_number
double extension
double discount_amount
double database_extension
double cost_of_goods_sold
char *credit_account_name

# PredictiveBooks Data Structure

## HOURLY_SERVICE_SALE

```
ENTITY *customer_entity =
        entity_new(
            full_name,
            street_address );
char *sale_date_time;
char *service_name;
char *service_description;
double hourly_rate;
double discount_amount;
double work_hours_database;
int estimated_hours;
double net_revenue_database;

char *hourly_service_sale_revenue_account_name(
        service_name,
        service_description );

LIST *hourly_service_work_list =
        work_hourly_list(
            full_name,
            street_address,
            sale_date_time,
            service_name,
            service_description );

double hourly_service_sale_work_hours =
        work_list_hours(
            hourly_service_work_list() );

double hourly_service_sale_net_revenue(
        hourly_rate,
        hourly_service_sale_work_hours,
        discount_amount )
{
        return (A x B) - C;
}
```

## SALE (4)

```
LIST *    fixed_service_sale_list
LIST *    hourly_service_sale_list
```

## FIXED_SERVICE_SALE

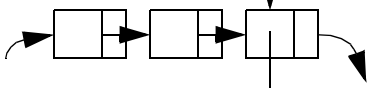
```
ENTITY *customer_entity =
        entity_new(
            full_name,
            street_address );
char *sale_date_time;
char *service_name;
double fixed_price;
double discount_amount;
double work_hours_database;
int estimated_hours;
double net_revenue_database;

char *fixed_service_sale_revenue_accouint_name(
        service_name );

LIST *fixed_service_sale_work_list =
        work_fixed_list(
            full_name,
            street_address,
            sale_date_time,
            service_name );

double fixed_service_sale_work_hours =
        work_list_hours(
            fixed_service_sale_work_list() );

double fixed_service_sale_net_revenue(
        fixed_price,
        discount_amount )
{
        return A - B;
}
```

## WORK

```
ENTITY *customer_entity = entity_new();
char *sale_date_time;
char *service_name;
char *service_description;
char *begin_date_time;
char *end_date_time;
char *login_name;
char *activity_name;

double work_hours(
        begin_date_time,
        end_date_time );

double work_list_hours(
        work_list );
```

## PURCHASE (1)

```
ENTITY *vendor_entity =
        entity_fetch(
            full_name,
            street_address );

char *purchase_date_time;
double sales_tax;
double freight_in;
char *arrived_date_time;
char *program_name;
char *property_street_address;

double purchase_equipment_total(
        purchase_equipment_list() )
{
        return sum(
                A->
                quipment_purchase->
                purchase_price );
}

double Purchase_invoice_amount(
        purchase_equipment_total(),
        sales_tax,
        freight_in );

double purchase_fetch_amount_due(
        full_name,
        street_address,
        purchase_date_time );

double Purchase_amount_due(
            Purchase_invoice_amount(),
            vendor_payment_total() )
{
        return A - B;
}

LIST *purchase_equipment_list(
        full_name,
        street_address,
        purchase_date_time );

TRANSACTION *purchase_transaction(
        full_name,
        street_address,
        arrived_date_time,
        Purchase_invoice_amount(),
        purchase_asset_account_name(
            purchase_equipment_list() ),
        account_payable() );

void purchase_update(
        purchase_equipment_total(),
        Purchase_invoice_amount(),
        vendor_payment_total(),
        Purchase_amount_due(),
        purchase_transaction()->
            transaction_date_time,
        full_name,
        street_address,
        purchase_date_time );

/* Returns true transaction_date_time */
char *purchase_transaction_refresh(
        purchase_transaction()->
            transaction_amount,
        purchase_transaction()->
            journal_list,
        purchase_transaction->memo,
        full_name,
        street_address,
        purchase_transaction()->
            transaction_date_time );

LIST *purchase_journal_list(
        Purchase_invoice_amount(),
        purchase_asset_account_name(),
        account_payable() );
```



**EQUIPMENT_PURCHASE**

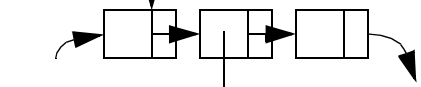
```
char *asset_name;
char *serial_number;
ENTITY *vendor_entity;
char *purchase_date_time;
char *service_placement_date;
double equipment_cost;
char *asset_account_name;
LIST *equipment_depreciation_list;
```

**ENTITY**

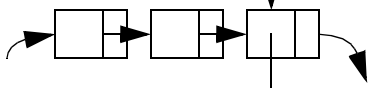
```
char *full_name;
char *street_address;
```

## PURCHASE (2)

```
double   sum_inventory_extension
double   sum_specific_inventory_extension
double   sum_supply_extension
double   sum_service_extension
double   sum_fixed_asset_extension
double   sum_prepaid_asset_extension
doulbe   sum_extension
double   database_sum_extension
double   sales_tax
double   freight_in
double   purchase_amount
double   sum_payment_amount

double   amount_due
double   database_amount_due
double   liability_payment_amount
char *   shipped_date
char *   database_shipped_date
char *   arrived_date_time
char *   database_arrived_date_time
LIST *   inventory_purchase_list
LIST *   specific_inventory_purchase_list
LIST *   service_purchase_list
LIST *   supply_purchase_list
LIST *   property_fixed_asset_purchase_list

LIST *   prepaid_asset_purchase_list

TRANSACTION *purchase_transaction(
         full_name,
         street_address,
         arrived_date_time,
         purchase_invoice_amount,
         purchase_asset_account_name(
              purchase_equipment_list() ),
         account_payable() );

char *property_street_address
```



**INVENTORY_PURCHASE**

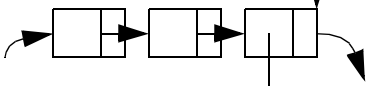


**PURCHASE_SUPPLY**

```
char *supply_name
int quantity
double unit_cost
double extension
double database_extension
char *account_name
```

```
enum title_passage_rule
title_passage_rule
```

FOB_shipping
FOB_destination
title_passage_rule_null

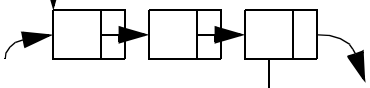


**PURCHASE_SERVICE**

```
char *account_name
double extension
```

**PURCHASE_SPECIFIC_INVENTORY**

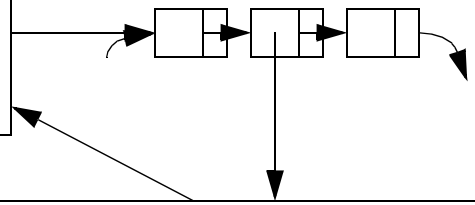
```
char *inventory_name
char *serial_number
double extension
double capitalized_extension
double database_capitalized_extension
```



**PURCHASE_PREPAID_ASSET**

## PURCHASE (3)

```
LIST *purchase_vendor_payment_list =
        vendor_payment_list(
            full_name,
            street_address,
            purchase_date_time );
```



## VENDOR_PAYMENT

```
ENTITY *vendor_entity =
        entity_fetch(
            full_name,
            street_address );

char *purchase_date_time;
char *payment_date_time;
double payment_amount;
int check_number;

PURCHASE *purchase;

LIST *vendor_payment_list(
        full_name,
        street_address,
        purchase_date_time );

LIST *vendor_payment_fetch_list(
        where );

VENDOR_PAYMENT *vendor_payment_new(
        full_name,
        street_address,
        purchase_date_time,
        payment_date_time );

VENDOR_PAYMENT *vendor_payment_fetch(
        full_name,
        street_address,
        purchase_date_time,
        payment_date_time );

char *vendor_payment_select();

char *vendor_payment_primary_where(
        full_name,
        street_address,
        purchase_date_time,
        payment_date_time );

VENDOR_PAYMENT *vendor_payment_parse(
        input );

LIST *vendor_payment_system_list(
        sys_string );

double vendor_payment_total(
        LIST *vendor_payment_list() )
{
        return sum(
                A->
                vendor_payment->
                payment_amount );
}

TRANSACTION *vendor_payment_transaction(
        full_name,
        street_address,
        payment_date_time,
        payment_amount,
        check_number,
        account_payable(),
        account_cash(),
        account_uncleared_checks() );

LIST *vendor_payment_journal_list(
        payment_amount,
        check_number,
        account_payable(),
        account_cash(),
        account_uncleared_checks() );

void vendor_payment_insert(
        full_name,
        street_address,
        purchase_date_time,
        payment_date_time,
        payment_amount,
        check_number );

VENDOR_PAYMENT *vendor_payment_seek(
        vendor_payment_list,
        payment_date_time );

char *vendor_payment_update_sys_string(
        void );
```

## EQUIPMENT_PURCHASE (1)

```
char *    asset_name;
char *    serial_number;
char *    service_placement_date;
double    equipment_cost;
int       estimated_useful_life_years;
int       estimated_useful_life_units;
int       estimated_residual_value;
int       declining_balance_n;
char *    depreciation_method;
double    tax_cost_basis;
char *    tax_recovery_period;
char *    disposal_date;
char *    equipment_account_name;
double    finance_accumulated_depreciation;
double    tax_accumulated_depreciation;
TAX_RECOVERY *tax_recovery;

ENTITY *vendor_entity =
          vendor_fetch(
               full_name,
               street_address );

EQUIPMENT_PURCHASE *
          equipment_purchase_new(
               asset_name,
               serial_number,
               full_name,
               street_address,
               purchase_date_time );

EQUIPMENT_PURCHASE *
          equipment_purchase_fetch(
               asset_name,
               serial_number,
               full_name,
               street_address,
               purchase_date_time );

EQUIPMENT_PURCHASE *
          equipment_purchase_parse(
               input );

char *equipment_purchase_select(
          void );

FILE *equipment_purchase_update_open(
          void );

void equipment_purchase_update(
          finance_accumulated_depreciation,
          tax_accumulated_depreciation,
          asset_name,
          serial_number,
          full_name,
          street_address,
          purchase_date_time );

LIST *equipment_purchase_list(
          full_name,
          street_address,
          purchase_date_time );
```

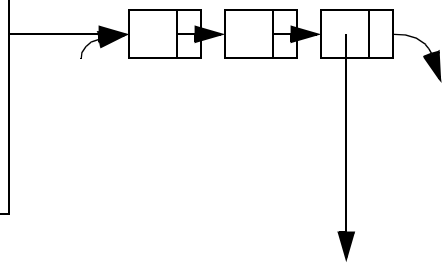
## TAX_RECOVERY

| | | | |
|---|---|---|---|
| char * | service_placecment_date | char * | disposal_date |
| double | tax_cost_basis | int | tax_year |
| char * | tax_recovery_period | double | recovery_amount |
| double | tax_recovery_period_years | double | recovery_percent |

## ENTITY

## EQUIPMENT_PURCHASE (2)

```
LIST *depreciation_list =
        depreciation_list(
            asset_name,
            serial_number,
            full_name,
            street_address,
            purchase_date_time );

DEPRECIATION *equipment_purchase_depreciation(
        equipment_purchase,
        depreciation_date,
        prior_depreciation_date,
            transaction_date_time,
            units_produced );

LIST *equipment_purchase_depreciation_list(
        equipment_purchase_list );

/* Returns equipment_purchase_list with */
/* equipment_purchase->depreciation_list set. */
LIST *equipment_purchase_list_depreciate(
        equipment_purchase_list(),
        depreciation_date,
        boolean set_depreciation_date );
```



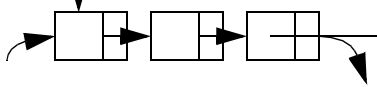
## DEPRECIATION

```
char *asset_name;
char *serial_number;

ENTITY *vendor_entity =
        entity_new(
            full_name,
            street_address ) );

char *purchase_date_time;
char *depreciation_date;

double depreciation_accumulated_depreciation(
        prior_accumulated_depreciation,
        depreciation_amount() );

double depreciation_amount(
        depreciation_method,
        equipment_cost,
        estimated_residual_value,
        estimated_useful_life_years,
        estimated_useful_life_units,
        declining_balance_n,
        prior_depreciation_date,
        depreciation_date_string,
        finance_accumulated_depreciation,
        service_placement_date,
        units_produced );

double depreciation_straight_line(
        equipment_cost,
        estimated_residual_value,
        estimated_useful_life_years,
        prior_depreciation_date,
        depreciation_date,
        finance_accumulated_depreciation );

double depreciation_sum_of_years_digits(
        equipment_cost,
        estimated_residual_value,
        estimated_useful_life_years,
        prior_depreciation_date,
        depreciation_date,
        finance_accumulated_depreciation,
        service_placement_date );

double depreciation_double_declining_balance(
        equipment_cost,
        estimated_residual_value,
        estimated_useful_life_years,
        prior_depreciation_date,
        depreciation_date,
        finance_accumulated_depreciation );

double depreciation_n_declining_balance(
        equipment_cost,
        estimated_residual_value,
        estimated_useful_life_years,
        prior_depreciation_date,
        depreciation_date,
        finance_accumulated_depreciation,
        n );

double depreciation_units_of_production(
        equipment_cost,
        estimated_residual_value,
        estimated_useful_life_units,
        units_produced,
        finance_accumulated_depreciation );

double depreciation_fraction_of_year(
        prior_depreciation_date,
        depreciation_date );

double depreciation_amount_total(
        depreciation_list() );

LIST *depreciation_list(
        asset_name,
        serial_number,
        full_name,
        street_address,
        purchase_date_time );

TRANSACTION *depreciation_transaction(
        full_name,
        street_address,
        transaction_date_time,
        depreciation_amount,
        account_depreciation_expense(),
        account_accumulated_depreciation() );

LIST *depreciation_transaction_list(
        depreciation_list );
```

## PURCHASE_PREPAID_ASSET

| | | | | | |
|---|---|---|---|---|---|
| char * | full_name | char * | asset_account_name | double | accumulated_accrual |
| char * | street_address | char * | expense_account_name | | |
| char * | purchase_date_time | double | extension | | |
| char * | asset_name | double | accrual_period_years | | |

LIST *
accrual_list



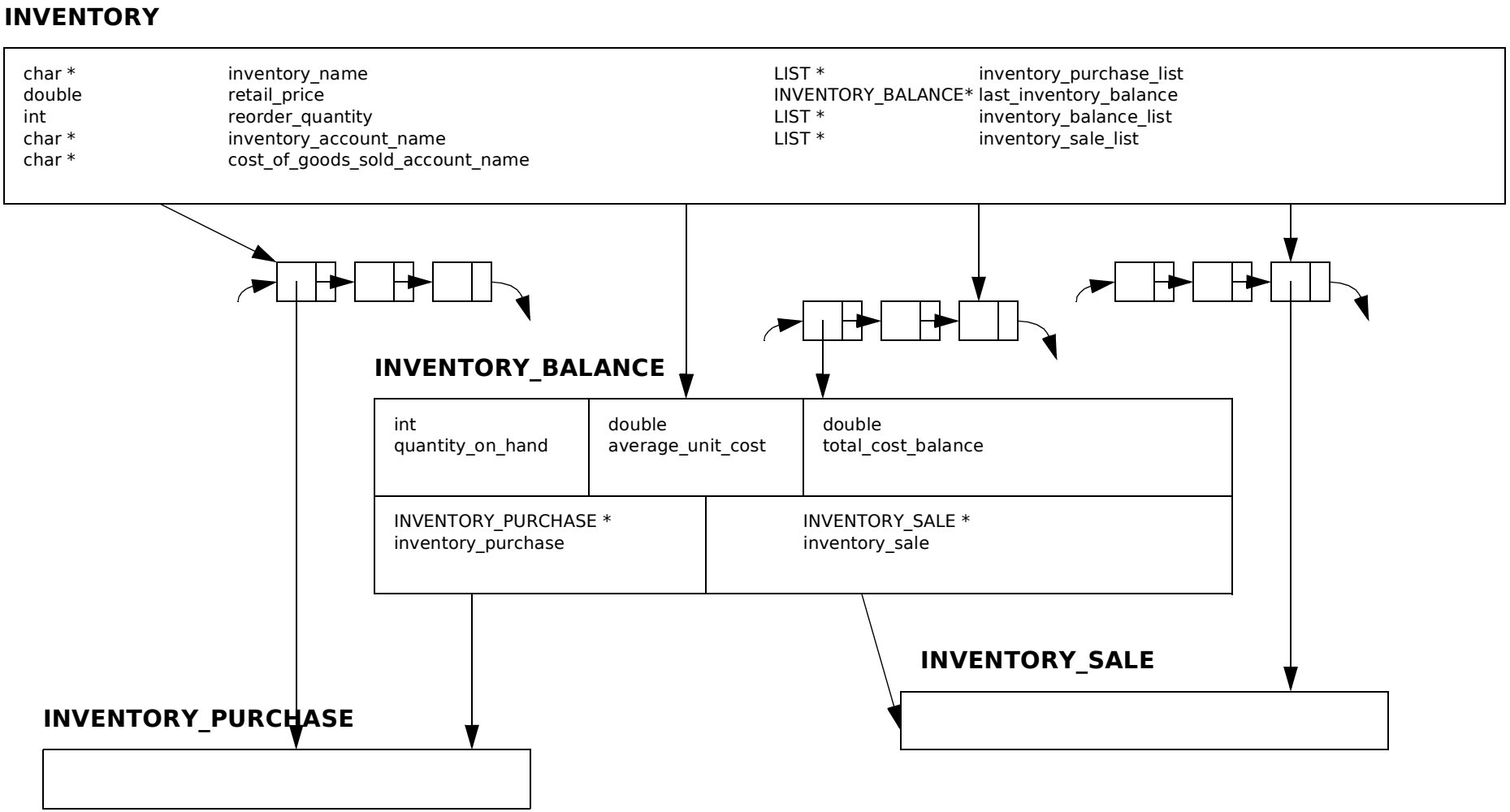
## ACCRUAL

| | | | |
|---|---|---|---|
| char * | full_name | double | accrual_amount |
| char * | street_address | double | database_accrual_amount |
| char * | purchase_date_time | char * | transaction_date_time |
| char * | asset_name | char * | database_transaction_date_time |
| char * | accrual_date | TRANSACTION *transaction | |

## VENDOR_PAYMENT

| | | | | |
|---|---|---|---|---|
| char * payment_date_time | double payment_amount | char * transaction_date_time | char * database_transaction_date_time | TRANSACTION * transaction |

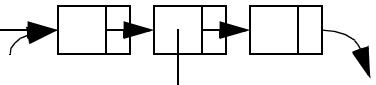
## TRANSACTION


INVENTORY
char * inventory_name
double retail_price
int reorder_quantity
char * inventory_account_name
char * cost_of_goods_sold_account_name
LIST * inventory_purchase_list
INVENTORY_BALANCE* last_inventory_balance
LIST * inventory_balance_list
LIST * inventory_sale_list
INVENTORY_BALANCE
int quantity_on_hand
double average_unit_cost
double total_cost_balance
INVENTORY_PURCHASE * inventory_purchase
INVENTORY_SALE * inventory_sale
INVENTORY_SALE
INVENTORY_PURCHASE

# PredictiveBooks Data Structure

## INVENTORY_PURCHASE

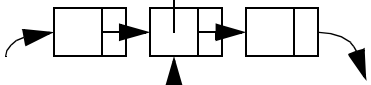
| | | | |
|---|---|---|---|
| char * | full_name | int | database_arrived_quantity |
| char * | street_address | int | missing_quantity |
| char * | purchase_date_time | int | quantity_on_hand |
| char * | inventory_name | int | database_quantity_on_hand |
| int | ordered_quantity | int | layer_consumed_quantity |
| double | unit_cost | double | average_unit_cost |
| double | capitalized_unit_cost | double | database_average_unit_cost |
| double | database_capitalized_unit_cost | char * | arrived_date_time |
| double | extension | char * | inventory_account_name |
| double | database_extension | LIST * | inventory_purchase_return_list |
| int | arrived_quantity | | |



## INVENTORY_PURCHASE_RETURN

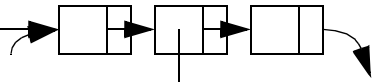
| | |
|---|---|
| char * | return_date_time |
| int | returned_quantity |
| double | sales_tax |
| char * | transaction_date_time |
| char * | database_transaction_date_time |
| TRANSACTION * | transaction |

## TRANSACTION



## INVENTORY_SALE

| | | | |
|---|---|---|---|
| char * | full_name | double | database_extension |
| char * | street_address | double | cost_of_goods_sold |
| char * | sale_date_time | double | database_cost_of_goods_sold |
| char * | inventory_name | char * | completed_date_time |
| int | sold_quantity | char * | inventory_account_name |
| double | retail_price | char * | cost_of_goods_sold_account_name |
| double | discount_amount | LIST * | inventory_sale_return_list |
| double | extension | LIST * | layer_inventory_purchase_list |



## INVENTORY_SALE_RETURN

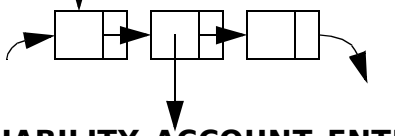
| | |
|---|---|
| char * | return_date_time |
| int | returned_quantity |
| char * | transaction_date_time |
| char * | database_transaction_date_time |
| TRANSACTION * | transaction |

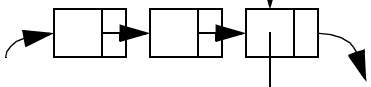
## TRANSACTION

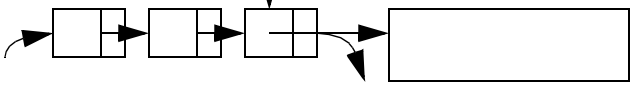
# PredictiveBooks Data Structure

## LIABILITY

```
/* Input */
int starting_check_number;
LIST *liability_account_entity_list();
LIST *liability_current_account_list();

LIST *input_entity_list =
        entity_full_street_list(
            full_name_list,
            street_address_list );

/* Process */

LIST *liability_tax_redirect_account_list(
        liability_current_account_list(),
        liability_account_entity_list() );

LIST *liability_entity_list(
        liability_tax_redirect_account_list()
            /* liability_account_list */,
        input_entity_list,
        dialog_box_payment_amount );

LIST *liability_after_balance_zero_entity_list(
        liability_entity_list(),
        liability_tax_redirect_account_list()
            /* liability_account_list */ );

LIST *liability_steady_state_entity_list(
        liability_after_balance_zero_entity_list() );

LIST *liability_transaction_list(
        liability_entity_list_steady_state(),
        liability_credit_account_name(
            starting_check_number ),
        account_loss() );
```



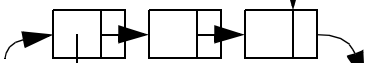


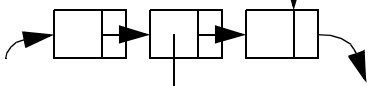


## TRANSACTION

## LIABILITY_ACCOUNT_ENTITY

```
char *account_name;
ENTITY *entity;
```



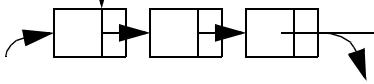


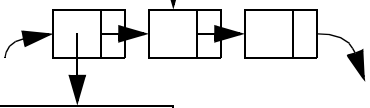
## ENTITY

```
char *full_name;
char *street_address;
int check_number;
double dialog_box_payment_amount;

LIST *liability_after_balance_zero_journal_list(
        liability_account_list(),
        full_name,
        street_address );

char *liability_entity_debit_account_name(
        account->account_name );

double liability_entity_amount_due(
        liability_entity_journal_list() );

double liability_entity_payment_amount(
        dialog_box_payment_amount,
        liability_entity_amount_due() );

double liability_entity_loss_amount(
        dialog_box_payment_amount,
        liability_entity_payment_amount() )
{
        return A - B;
}

ENTITY *liability_steady_state_entity(
        entity,
        liability_entity_journal_list() );
```

## ACCOUNT

```
char *account_name;

LIST *transaction_after_balance_zero_journal_list(
        account_name );
```





## JOURNAL

```
char *account_name;
char *full_name;
char *street_address;
double debit_amount;
double credit_amount;
```

# PredictiveBooks Data Structure



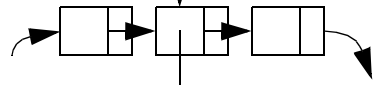
## EMPLOYEE

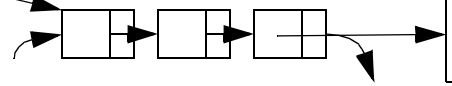
char *full_name
char *street_address
double hourly_wage
double period_salary
double commission_sum_extension_percent
double gross_pay_year_to_date
double database_gross_pay_year_to_date
double net_pay_year_to_date
double database_net_pay_year_to_date
char * federal_marital_status
int federal_withholding_allowances
int federal_withholding_additional_period_amount
boolean state_withholding_exempt
char *state_marital_status
int state_withholding_allowances
int state_itemized_deduction_allowances
int retirement_contribution_plan_employee_period_amount
int retirement_contribution_plan_employer_period_amount
int health_insurance_employee_period_amount
int health_insurance_employer_period_amount
double union_dues_period_amount
char *terminated_date

LIST *
employee_work_day_list

LIST *
employee_work_period_list

LIST *
customer_sale_list





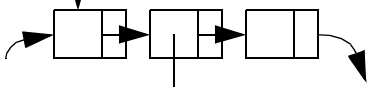
## CUSTOMER_SALE

## EMPLOYEE_WORK_PERIOD

char *full_name
char *street_address
int payroll_year
int payroll_period_number
char *begin_work_date
char *end_work_date
double regular_work_hours
double overtime_work_hours
double commission_sum_extension
double gross_pay
double net_pay
double payroll_tax_amount
double federal_tax_withholding_amount
double state_tax_withholding_amount
double social_security_employee_tax_amount
double social_security_employer_tax_amount
double medicare_employee_tax_amout
double medicare_employer_tax_amount
int
retirement_contribution_plan_employee_amount
int retirement_contribution_plan_employer_amount
int health_insurance_employee_amount
int health_insurance_employer_amount
double federal_unemployment_tax_amount
double state_unemployment_tax_amount
int union_dues_amount
char *transaction_date_time

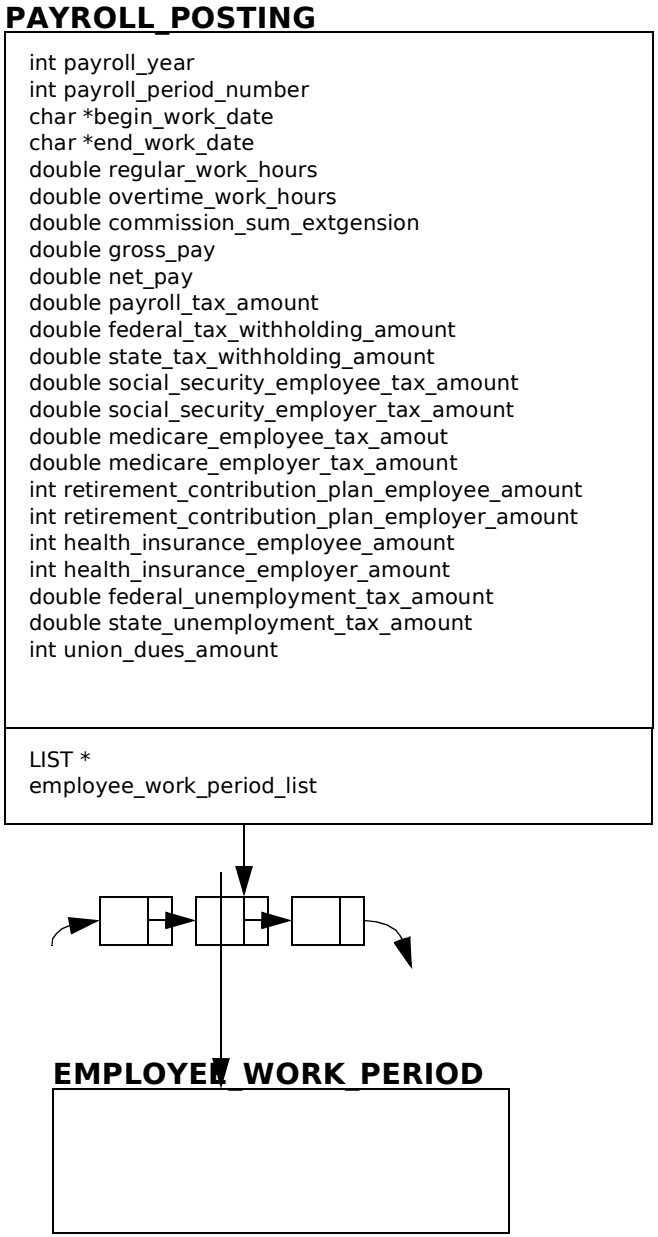
TRANSACTION *
transaction

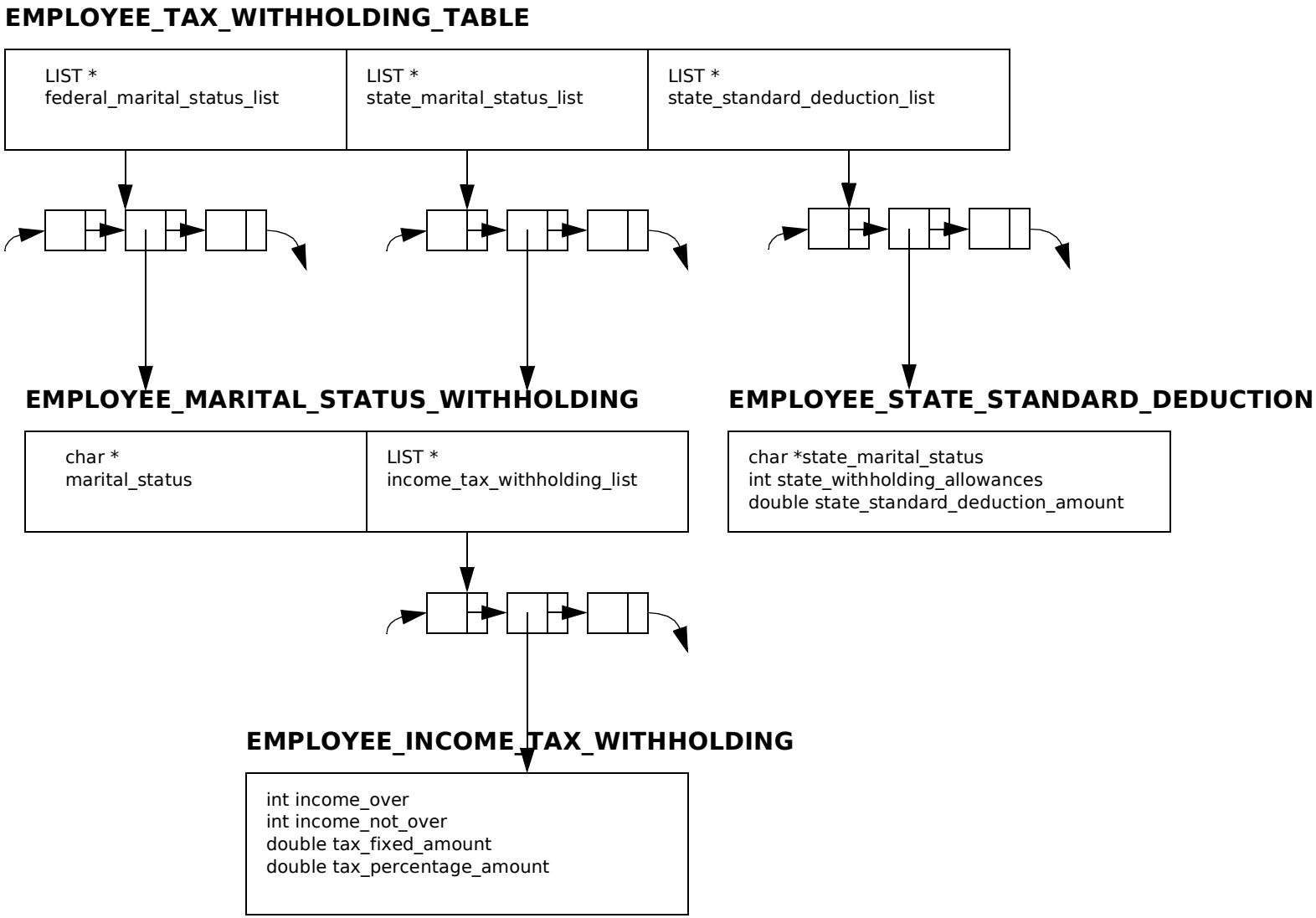
## TRANSACTION



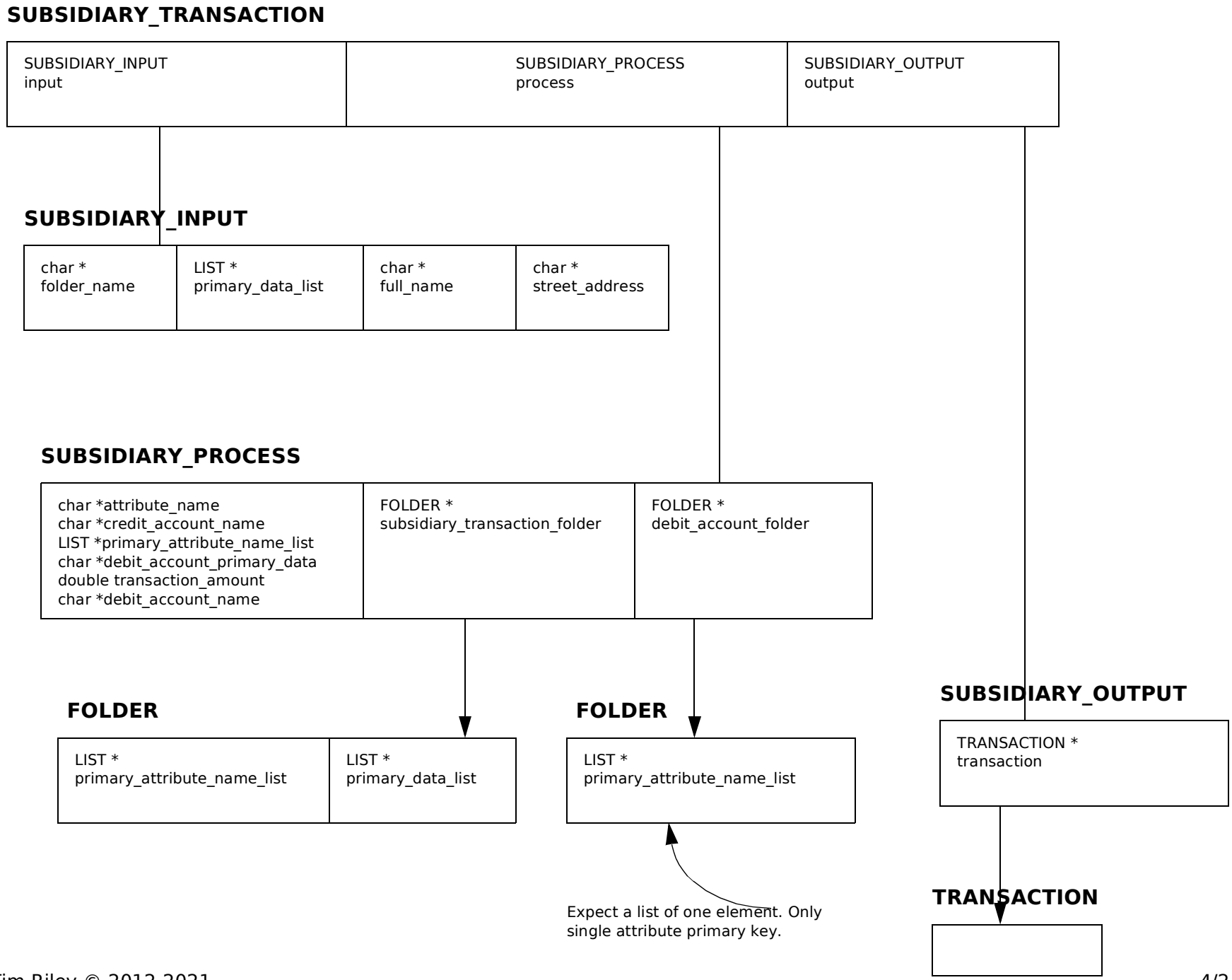
## EMPLOYEE_WORK_DAY

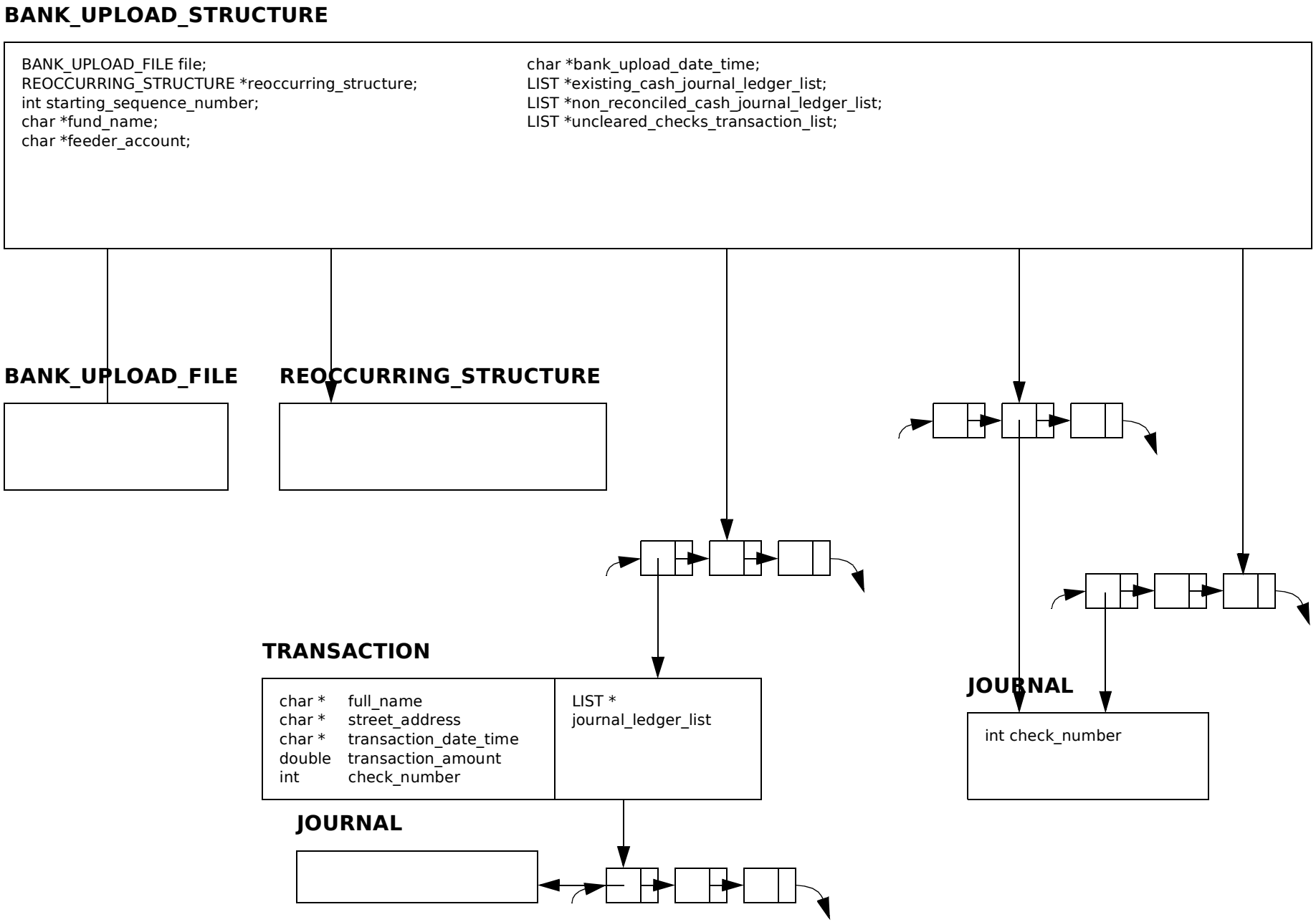
char *begin_work_date_time
char *end_work_date_time
double employee_work_hours
double database_employee_work_hours
boolean overtime_work_day

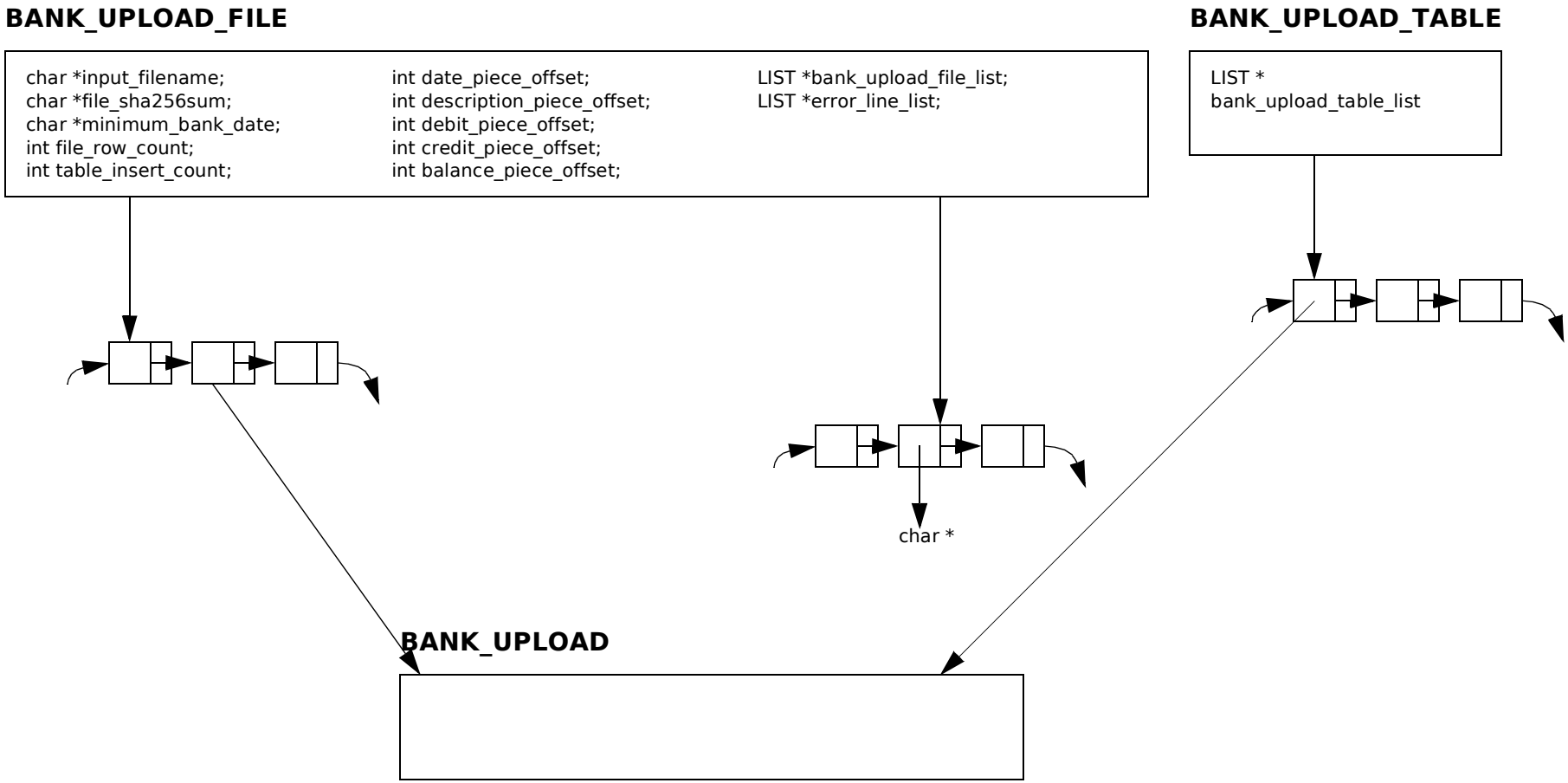

PAYROLL_POSTING
int payroll_year
int payroll_period_number
char *begin_work_date
char *end_work_date
double regular_work_hours
double overtime_work_hours
double commission_sum_extgension
double gross_pay
double net_pay
double payroll_tax_amount
double federal_tax_withholding_amount
double state_tax_withholding_amount
double social_security_employee_tax_amount
double social_security_employer_tax_amount
double medicare_employee_tax_amout
double medicare_employer_tax_amount
int retirement_contribution_plan_employee_amount
int retirement_contribution_plan_employer_amount
int health_insurance_employee_amount
int health_insurance_employer_amount
double federal_unemployment_tax_amount
double state_unemployment_tax_amount
int union_dues_amount
LIST *
employee_work_period_list
EMPLOYEE_WORK_PERIOD

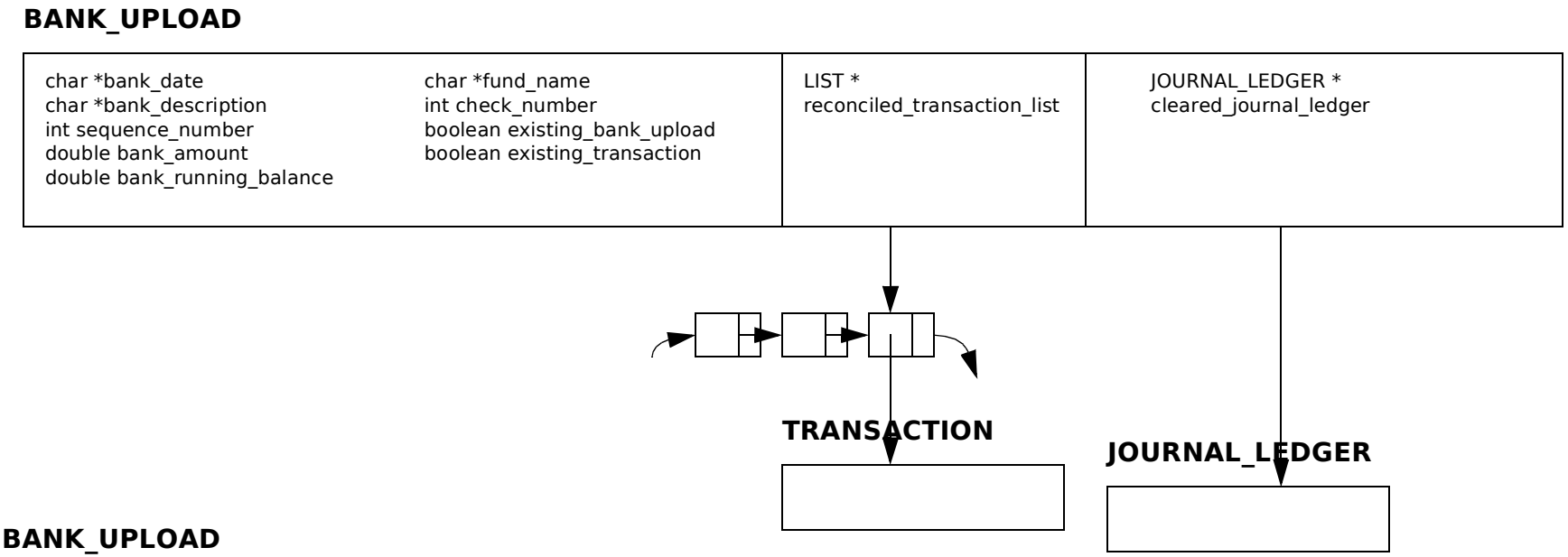

EMPLOYEE_TAX_WITHHOLDING_TABLE
LIST *
federal_marital_status_list
LIST *
state_marital_status_list
LIST *
state_standard_deduction_list
EMPLOYEE_MARITAL_STATUS_WITHHOLDING
char *
marital_status
LIST *
income_tax_withholding_list
EMPLOYEE_STATE_STANDARD_DEDUCTION
char *state_marital_status
int state_withholding_allowances
double state_standard_deduction_amount
EMPLOYEE_INCOME_TAX_WITHHOLDING
int income_over
int income_not_over
double tax_fixed_amount
double tax_percentage_amount

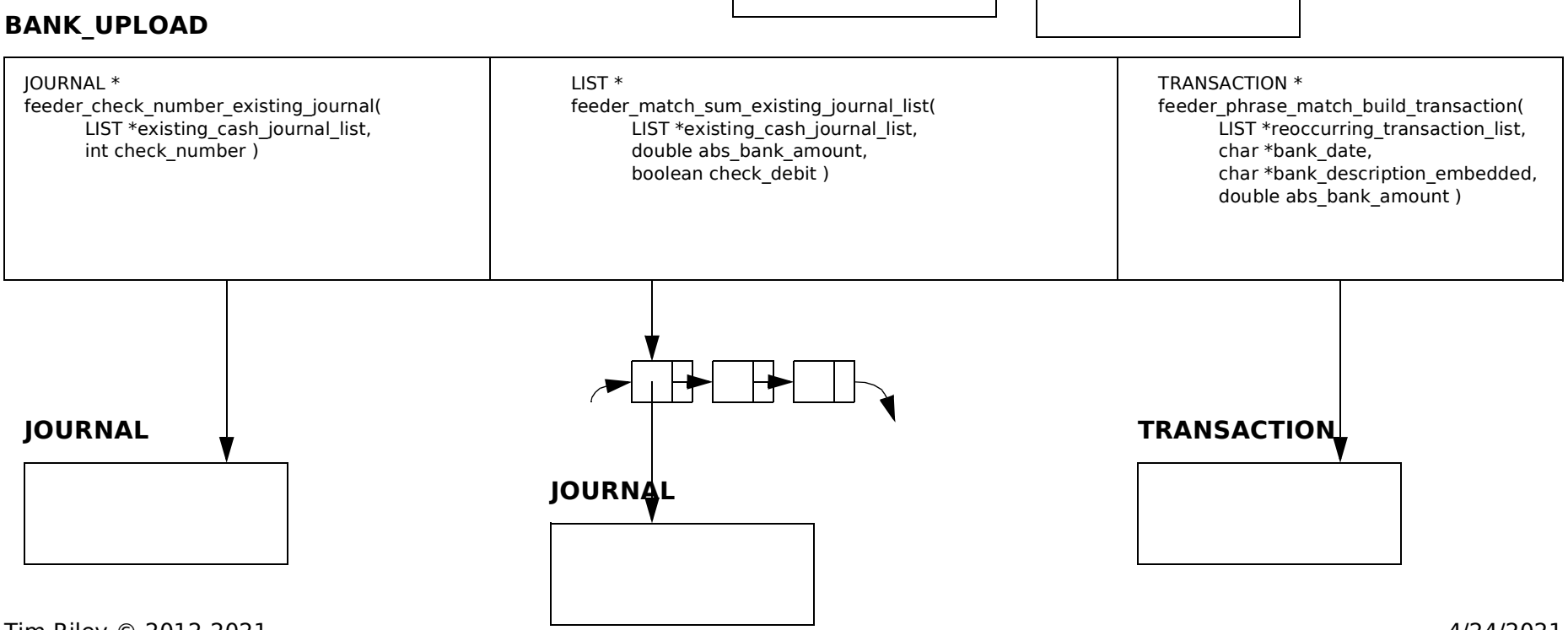
# PredictiveBooks Data Structure

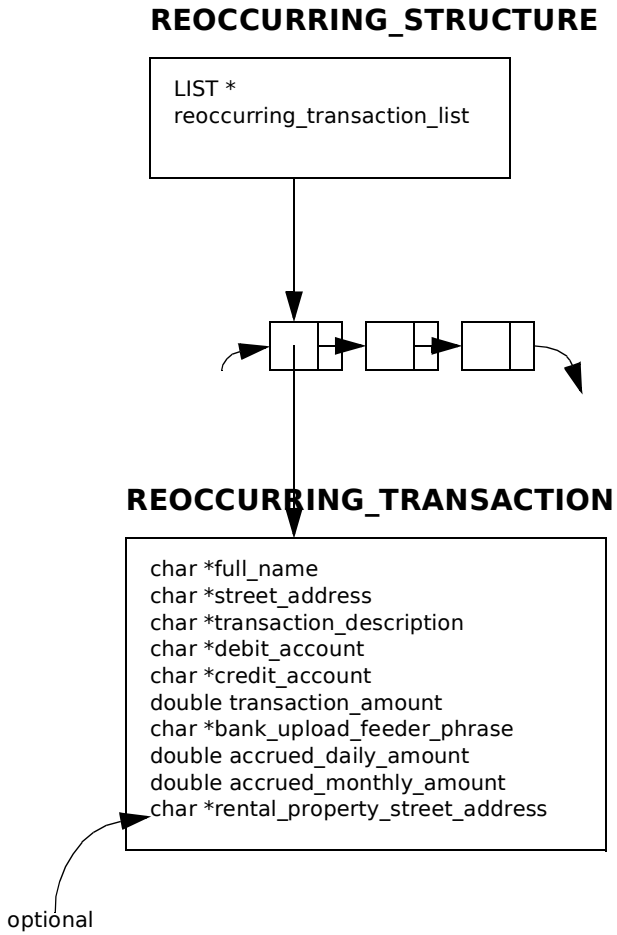



BANK_UPLOAD_STRUCTURE
BANK_UPLOAD_FILE file;
REOCCURRING_STRUCTURE *reoccurring_structure;
int starting_sequence_number;
char *fund_name;
char *feeder_account;
char *bank_upload_date_time;
LIST *existing_cash_journal_ledger_list;
LIST *non_reconciled_cash_journal_ledger_list;
LIST *uncleared_checks_transaction_list;
BANK_UPLOAD_FILE
REOCCURRING_STRUCTURE
TRANSACTION
char * full_name
char * street_address
char * transaction_date_time
double transaction_amount
int check_number
LIST *
journal_ledger_list
JOURNAL
int check_number
JOURNAL


BANK_UPLOAD_FILE
char *input_filename;
char *file_sha256sum;
char *minimum_bank_date;
int file_row_count;
int table_insert_count;
int date_piece_offset;
int description_piece_offset;
int debit_piece_offset;
int credit_piece_offset;
int balance_piece_offset;
LIST *bank_upload_file_list;
LIST *error_line_list;
BANK_UPLOAD_TABLE
LIST *
bank_upload_table_list
char *
BANK_UPLOAD


BANK_UPLOAD
char *bank_date
char *bank_description
int sequence_number
double bank_amount
double bank_running_balance
char *fund_name
int check_number
boolean existing_bank_upload
boolean existing_transaction
LIST *
reconciled_transaction_list
JOURNAL_LEDGER *
cleared_journal_ledger
TRANSACTION
JOURNAL_LEDGER


BANK_UPLOAD
JOURNAL *
feeder_check_number_existing_journal(
LIST *existing_cash_journal_list,
int check_number )
LIST *
feeder_match_sum_existing_journal_list(
LIST *existing_cash_journal_list,
double abs_bank_amount,
boolean check_debit )
TRANSACTION *
feeder_phrase_match_build_transaction(
LIST *reoccurring_transaction_list,
char *bank_date,
char *bank_description_embedded,
double abs_bank_amount )
JOURNAL
JOURNAL
TRANSACTION


REOCCURRING_STRUCTURE
LIST *
reoccurring_transaction_list
REOCCURRING_TRANSACTION
char *full_name
char *street_address
char *transaction_description
char *debit_account
char *credit_account
double transaction_amount
char *bank_upload_feeder_phrase
double accrued_daily_amount
double accrued_monthly_amount
char *rental_property_street_address
optional

## PAYPAL

```
/* Usage */
PAYPAL *paypal_fetch(
        spreadsheet_filename,
        date_label );

/* Process */
SPREADSHEET *spreadsheet =
        spreadsheet_fetch(
            spreadsheet_filename,
            date_label );
```

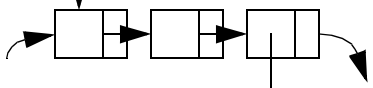
## SPREADSHEET

## PAYPAL_DATASET

```
/* Input */
char *date_A;
char *time_B;
char *timezone_C;
char *full_name_D;
char *transaction_type_E;
char *transaction_status_F;
char *currency_G;
char *gross_revenue_H;
char *transaction_fee_I;
char *net_revenue_J;
char *from_email_address_K;
char *to_email_address_L;
char *transaction_ID_M;
char *shipping_address_N;
char *address_status_O;
char *item_title_P;
char *item_ID_Q;
char *shipping_handling_amount_R;
char *insurance_amount_S;
char *sales_tax_T;
char *option_1_name_U;
char *option_1_value_V;
char *option_2_name_W;
char *option_2_value_X;
char *reference_txn_ID_Y;
char *invoice_number_Z;
char *custom_number_AA;
char *quantity_AB;
char *receipt_ID_AC;
char *balance_AD;
char *address_line_1_AE;
char *address_2_district_neighborhood_AF;
char *town_city_AG;
char *state_province_AH;
char *zip_code_AI;
char *country_AJ;
char *contact_phone_number_AK;
char *subject_AL;
char *note_AM;
char *country_code_AN;
char *balance_impact_AO;
```

## SPREADSHEET

```
/* Input */
char *spreadsheet_filename;
char *date_label;

/* Process */
LIST *spreadsheet_column_list(
        spreadsheet_filename,
        date_label );

char *maximum_date;

char *spreadsheet_minimum_date(
        &maximum_date,
        spreadsheet_filename );

char *spreadsheet_heading_data(
        spreadsheet_column_list(),
        input_row,
        heading );
```



## SPREADSHEET_COLUMN

```
/* Usage */
SPREADSHEET_COLUMN *spreadsheet_column_new(
        heading,
        piece_offset );

/* Process */
SPREADSHEET_COLUMN *spreadsheet_column_heading_seek(
        spreadsheet_column_list,
        heading );
```

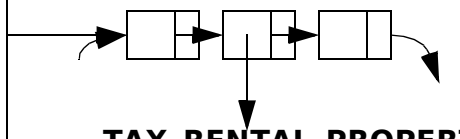
## SPREADSHEET_UPLOAD_EVENT

```
/* Input */
char *spreadsheet_filename;

/* Process */
char *spreadsheet_upload_date_time =
        date_time19_now();

char *spreadsheet_upload_file_sha256sum(
        spreadsheet_filename );
```

## TAX

```
/* Usage */
TAX *tax_new(
        char *tax_form_string,
        int tax_year );

/* Input */
char *tax_form_string;
int tax_year;

/* Attributes */
```

```
TAX_FORM *tax_form =
        tax_form_fetch();

TAX_RECOVERY *tax_recovery;

LIST *tax_rental_property_list =
        tax_rental_property_list_fetch(
            tax_form_string,
            tax_year );
```



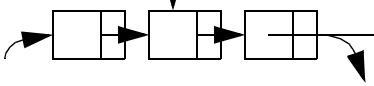
## TAX_RENTAL_PROPERTY

## TAX_FORM

```
/* Usage */
TAX_FORM *tax_form_fetch(
        char *tax_form_string,
        int tax_year,
        boolean fetch_line_list,
        boolean fetch_account_list,
        boolean fetch_journal_list );

/* Attributes */
char *tax_form_string;

/* Input */
int tax_year;

/* Process */
LIST *tax_form_line_list(
        tax_form_string,
        tax_year,
        fetch_account_list,
        fetch_journal_list );
```

## TAX_RECOVERY

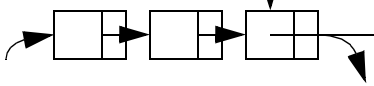
```
double fixed_asset_recovery_amount;
double prior_fixed_asset_recovery_amount;
double property_recovery_amount;
double prior_property_recovery_amount;
double total_recovery_amount;
```



## TAX_FORM_LINE

```
/* Usage */
LIST *tax_form_line_list(
        char *tax_form_string,
        int tax_year,
        boolean fetch_account_list,
        boolean fetch_journal_list );

/* Attributes */
char *tax_form_string;
char *tax_form_line_string;
char *tax_form_description;
boolean itemize_accounts;

/* Input */
int tax_year;
```

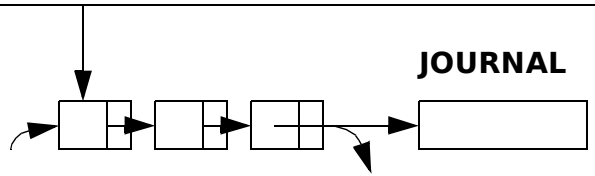
```
/* Process */
LIST *tax_form_line_account_list(
        tax_form_string,
        tax_form_line_string,
        tax_year,
        fetch_journal_list );

/* Operations */
void tax_form_line_list_steady_state(
        tax_form_line_list );

double tax_form_line_total(
        tax_form_line_account_list() );
```



## TAX_FORM_LINE_ACCOUNT

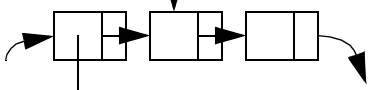
## TAX_FORM_LINE_ACCOUNT

```
/* Usage */
LIST *tax_form_line_account_list(
        char *tax_form_string,
        char *tax_form_line,
        int tax_year,
        boolean fetch_journal_list );

/* Attributes */
char *tax_form_string;
char *tax_form_line;
char *account_name;

/* Process */
ACCOUNT *account =
        /* Sets account->accumulate_debit */
        account_fetch(
            account_name );

boolean current_liability =
        subclassification_current_liability(
            account->subclassification_name );

LIST *journal_list =
        journal_year_list(
            tax_year,
            account_name,
            1 /* fetch_transaction_memo */ );

/* Operations */
double tax_form_line_account_total(
        journal_list,
        account->accumulate_debit,
        current_liability );
```



## TAX_RENTAL_PROPERTY

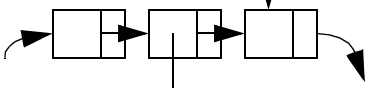
```
/* Usage */
LIST *tax_rental_property_list_fetch(
        char *tax_form_string,
        int tax_year );

/* Attributes */
char  *property_street_address;

/* Process */
LIST *tax_rental_property_string_list(
        tax_rental_property_list );

LIST *tax_form_line_rental_list(
        tax_form_line_list(),
        tax_rental_property_string_list );

double recovery_amount;
double revenue_total [23a];
double mortgage_interest_total [23c];
double depreciation_total [23d];
double expense_total [23e];
double positive_net_income [24];
double loss_total [25];
double tax_rental_property_total();
```



## TAX_FORM_LINE_RENTAL

```
char *tax_form_line;
char *tax_form_description;

enum tax_form_line_category;
LIST *tax_form_line_account_list();

LIST *rental_property_list;
double tax_form_line_rental_property_total();
```

revenue
mortgage_interest_paid
depreciation_expense
operating_expense
loss

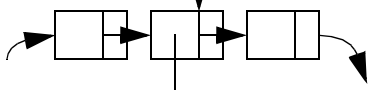


## TAX_FORM_LINE_ACCOUNT

## INVESTMENT_ACCOUNT

```
ENTITY *financial_entity =
        entity_new(
            full_name,
            street_address );

char *account_number;
char *investment_classification;
char *investment_purpose;

LIST *investment_account_balance_list(
```

```
full_name,
street_address,
account_number );
```



## ACCOUNT_BALANCE

```
char *date
double balance;
double balance_change;
int balance_change_percent;
```

## TRANSACTION

## ACCOUNT_BALANCE_EQUITY

```
char *date_time
char *investment_operation
double share_price
doubel share_quantity_change
double share_quantity_balance
double cash_in
double market_value
double book_value_change
double book_value_balance
```

```
double moving_share_price
double unrealized_gain_balance
double unrealized_gain_change
double reaqlized_gain
boolean table_first_row
char *transaction_date_time
```

```
TRANSACTION *
transaction
```

# PredictiveBooks Data Structure

## DEPRECIATION_STRUCTURE

| | | |
|---|---|---|
| DEPRECIATION_DATE *<br>depreciation_date | DEPRECIATION_TAX_YEAR *<br>depreciation_tax_year | LIST *depreciation_fund_list |

## DEPRECIATION_FUND

| | | | |
|---|---|---|---|
| char * fund_name<br>char * depreciation_expense_account<br>char * accumulated_depreciation_account | | DEPRECIATION_TRANSACTION *<br>depreciation_transaction | DEPRECIATION_ASSET_LIST *<br>depreciation_asset_list |
| DEPRECIATION_AMOUNT *<br>depreciation_amount | | | |

## DEPRECIATION_AMOUNT

| | |
|---|---|
| double | purchase_fixed_asset_depreciation_amount |
| double | prior_fixed_asset_depreciation_amount |
| double | purchase_property_depreciation_amount |
| double | prior_property_depreciation_amount |

## DEPRECIATION_TRANSACTION

| |
|---|
| TRANSACTION *purchase_equipment_transaction |
| TRANSACTION *prior_fixed_asset_transaction |
| TRANSACTION *purchase_property_transaction |
| TRANSACTION *prior_property_transaction |

## DEPRECIATION_TAX_YEAR

| | |
|---|---|
| int | undo_fixed_asset_year |
| int | undo_fixed_prior_year |
| int | undo_property_year |
| int | undo_property_prior_year |
| int | max_undo_year |

## DEPRECIATION_ASSET_LIST

| | | | |
|---|---|---|---|
| LIST *<br>purchase_fixed_asset_list | LIST *<br>prior_fixed_asset_list | LIST *<br>purchase_property_list | LIST *<br>prior_property_list |

## DEPRECIATION_DATE

| | |
|---|---|
| char * | undo_fixed_asset_date |
| char * | prior_fixed_asset_date |
| char * | undo_fixed_prior_date |
| char * | prior_fixed_prior_date |
| char * | undo_property_date |
| char * | prior_property_date |
| char * | undo_property_prior_date |
| char * | prior_property_prior_date |
| char * | max_undo_date |

## EQUIPMENT_PURCHASE